Simple CTF — TryHackMe (Fácil)

Plataforma: TryHackMe **Dificultad:** Fácil **Categorías:** Reconocimiento de red, enumeración web, SQL injection, cracking de contraseñas, escalada de privilegios en Linux **Herramientas:** Nmap, Gobuster, curl, Python 3, John the Ripper, SSH

Índice

1. Resumen
2. Reconocimiento
3. Enumeración web
4. Identificación de la vulnerabilidad
5. Explotación — SQL Injection
6. Acceso por SSH
7. Escalada de privilegios
8. Lecciones aprendidas
9. Mitigaciones
10. Referencias

Resumen

Simple CTF es una máquina Linux con tres servicios expuestos: FTP, HTTP y SSH en un puerto no estándar. La aplicación web oculta un CMS con una vulnerabilidad de inyección SQL ciega basada en tiempo, que permite extraer credenciales de la base de datos sin autenticación. Con esas credenciales se accede por SSH, y la escalada de privilegios se logra a través de un permiso de sudo mal configurado sobre vim.

1. Reconocimiento

Escaneo de puertos bajo el 1000, para responder la primera pregunta de la sala:

```
nmap -p 0-1000 <IP>
```

Resultado: dos puertos abiertos en ese rango — FTP (21) y HTTP (80).

Escaneo completo de todos los puertos para confirmar si hay más servicios:

```
nmap -p- -T4 <IP>
```

Nota importante: el escaneo completo con `-T4` sobre VPN no detectó un tercer puerto abierto (SSH en un puerto no estándar), a pesar de existir. Esto se debe a que la agresividad del timing combinada con la latencia de la VPN genera falsos negativos en puertos que responden más lento. El puerto fue verificado individualmente una vez que se supo de su existencia:

```
nmap -p <puerto_ssh> -sV --max-retries 3 <IP>
```

Lección: nunca confiar ciegamente en un solo escaneo completo. Cuando el contexto sugiere que puede haber más servicios, vale la pena verificar puertos específicos de forma individual.

Detección de versiones en los tres puertos confirmados:

```
nmap -sV -p 21,80,<puerto_ssh> <IP>
```

Puerto	Servicio	Versión
21	FTP	vsftpd 3.0.3
80	HTTP	Apache 2.4.18 (Ubuntu)
`<puerto>`	SSH	OpenSSH 7.2p2 Ubuntu

2. Enumeración web

El puerto 80 mostraba la página por defecto de Apache — señal de que la aplicación real estaba en otra ruta. Enumeración con Gobuster:

```
gobuster dir -u http://<IP>/ -w /usr/share/wordlists/dirb/common.txt
```

Entre los resultados, un directorio con código 301 (redirección) llamado `/simple/` destacó como el único no perteneciente a una instalación estándar de Apache.

Revisión de `robots.txt`, que tenía un tamaño inusualmente grande (929 bytes) para ser un archivo estándar:

```
curl -s http://<IP>/robots.txt
```

El archivo contenía una entrada `Disallow` con el nombre exacto del software y su versión: **CMS Made Simple 5.0.1.3** (codificado en el nombre de la ruta). Esto es información de reconocimiento valiosa sin necesidad de entrar a la aplicación.

Inspección del código fuente de `/simple/` para confirmar la aplicación y su versión:

```
curl -s http://<IP>/simple/ | grep -i -E "generator|version|powered|copyright"
```

Confirmado: **CMS Made Simple versión 2.2.8**, visible tanto en la metaetiqueta `Generator` como en el pie de página de la aplicación.

3. Identificación de la vulnerabilidad

Con el nombre y versión exactos del software, la búsqueda de vulnerabilidades conocidas arrojó **CVE-2019-9053**: una inyección SQL ciega basada en tiempo no autenticada, a través del módulo News y el parámetro `m1_idlist`. Afecta a CMS Made Simple versión 2.2.9 y anteriores, lo que incluye la 2.2.8 instalada.

Qué es una inyección SQL ciega basada en tiempo: a diferencia de una SQLi clásica (donde el resultado de la consulta se muestra directamente en la página), esta variante no devuelve ningún dato visible. En cambio, el atacante inyecta condiciones que hacen que la base de datos espere un número determinado de segundos antes de responder si la condición es verdadera. Midiendo el tiempo de respuesta, es posible extraer información carácter por carácter.

4. Explotación — SQL Injection

Se utilizó un script público en Python 3 para automatizar la explotación del CVE-2019-9053. El script extrae carácter por carácter, en este orden: el salt de la contraseña, el nombre de usuario, el email y el hash de la contraseña del administrador.

```
python3 exploit.py -u http://<IP>/simple --crack -w
/usr/share/wordlists/rockyou.txt
```

Dependencia requerida:

```
pip3 install termcolor requests --break-system-packages
```

Nota sobre el tiempo: con un valor de `TIME=2` (2 segundos de espera por carácter), la extracción completa tardó varios minutos. Es normal: la técnica es lenta por diseño, ya que cada bit de información requiere al menos una petición HTTP con espera intencional.

El script extrajo los siguientes datos:

- **Salt:** `<valor_del_salt>`
- **Usuario:** `mitch`
- **Email:** `admin@admin.com`
- **Hash:** `<hash_md5>`

Y crackeó la contraseña usando `rockyou.txt` comparando `md5(salt + candidato)` contra el hash extraído.

5. Acceso por SSH

Con el usuario y contraseña obtenidos:

```
ssh -p <puerto_no_estandar> <usuario>@<IP>
```

La flag de usuario estaba en el directorio personal, legible directamente con `cat`.

6. Escalada de privilegios

Verificación estándar de permisos de sudo:

```
sudo -l
```

Resultado: (root) NOPASSWD: /usr/bin/vim

El usuario tiene permiso para ejecutar **vim** como root sin contraseña. Vim incluye la capacidad de ejecutar comandos del sistema operativo desde dentro del editor mediante el comando interno `!`. Esto convierte cualquier permiso de sudo sobre vim en una escalada de privilegios directa:

```
sudo vim -c ':!/bin/bash'
```

- `sudo vim`: abre vim con privilegios de root.
- `-c`: ejecuta un comando interno de vim al iniciarse.
- `':!/bin/bash'`: lanza una shell bash desde dentro de vim, heredando los privilegios de root con los que se abrió vim.

Confirmación:

```
whoami
# root
```

La flag de root estaba en `/root/root.txt`.

Lecciones aprendidas

- Un escaneo completo de puertos con `-T4` sobre VPN puede generar falsos negativos. Verificar puertos sospechosos de forma individual con `--max-retries` es más confiable.
- `robots.txt` puede revelar el nombre exacto y versión de software instalado — información de reconocimiento que normalmente requiere acceso a la aplicación.
- Filtrar el código fuente con `grep -i -E "generator|version|powered"` es la forma más rápida de identificar un CMS y su versión sin leer el HTML completo.
- La inyección SQL ciega basada en tiempo es lenta pero efectiva: extrae datos carácter por carácter midiendo tiempos de respuesta, sin necesitar ninguna salida visible de la base de datos.
- Un permiso de sudo sobre un programa con capacidad de ejecutar comandos del sistema (como vim, python, find, awk) equivale a acceso root directo. GTFOBins documenta estas técnicas para decenas de programas.
- El tipo de wordlist importa: listas de rutas web para Gobuster, listas de contraseñas para cracking de hashes.

Mitigaciones (perspectiva defensiva)

- Actualizar CMS Made Simple a una versión posterior a 2.2.9, donde este CVE está parcheado.
- Usar consultas parametrizadas (prepared statements) en vez de concatenación directa de input del usuario en consultas SQL.
- No exponer información de versión del software en metaetiquetas, pies de página ni archivos públicos como `robots.txt`.
- Auditar los permisos de sudo regularmente: ningún usuario no privilegiado debería tener sudo sobre editores de texto, intérpretes de lenguajes, o cualquier programa listado en GTFOBins.
- Usar contraseñas robustas para cuentas de administrador de CMS — una contraseña de 6 caracteres como la de esta sala es crackeable en segundos con una wordlist estándar.

Referencias

- TryHackMe — Simple CTF
- CVE-2019-9053 — NVD
- GTFOBins — vim
- CVE-2019-9053 exploit